

DJANGO VIEWS

Django view's do the bulk of the work in implementing the actual functionality of the website. The more complex a website, likely the more complex its views

In the previous chapters we were very roughly introduced to a very basic view. That simple view that we — in our inestimable hubris — named ‘test_view’ it did the job for the time, it returned a simple response for whatever URL that linked to it. It's the view we deserved, but not the one we need right now.

The view we need right now is one that can display an article on the page. Django supports two ways of doing this. One is to use function-based views, where a URL is mapped to a function, the function receives the parameters from the URL, uses those parameters to generate HTML code to display to the user, and then returns that code as a response.

The other way is to use class-based views. These have been around for a long time, but initially Django only supported function-based views. The advantage of class-based views is that Django ships with many generic view classes that can handle a lot of common use cases such as displaying a list of items, or displaying details about a single item etc.

We'll first see how we can make this simple article details view using both a function and a class so we can compare how both work.

Comparing Function-Based and Class-Based Views

We'll begin with a function-based view for the article detail view. Remember